

Практическая работа №1

Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса.

Персональные параметры (заполните ДО начала работы):

Параметр	Как определить	Ваше значение
Номер ИСУ	Можно взять из ЛК	—
Персональный порт	5430 + последние 2 цифры ису Пример: $5430 + 45 = 5475$	—
Персональный префикс	Фамилия латиницей в нижнем регистре (например, ivanov)	—

!!!Важно!!! Все команды должны использовать ваши персональные значения. Отчёты с портом 5432 или без префикса <фамилия> — автоматически получают 0 баллов за соответствующие этапы.

Этап 1. Верификация окружения и архитектуры

Задание 1.1 — Проверьте установку Docker:

```
docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
```

Если на вашем ПК отсутствует Docker, необходимо его установить: <https://www.docker.com/get-started/>. После выполните команду выше.

docker version — выводит версии клиентской (docker CLI) и серверной (dockerd — демон Docker) частей.

--format — шаблонизатор Go для кастомизации вывода. `{{.Client.Version}}` и `{{.Server.Version}}` — поля из JSON-ответа.

Эти команды используются, чтобы убедиться, что демон Docker запущен и клиент может с ним взаимодействовать.

Запишите в отчет:

1. Версию клиента и сервера
2. Вывод команды `docker info --format "{{.OSType}}/{{.Architecture}}"`

Задание 1.2 — Исследуйте состояние локального хранилища:

```
docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
```

docker system df — аналог `df` в Linux, но для ресурсов Docker.

Показывает использование диска для 4 типов объектов:

1. Images — образы (слои файловой системы)
2. Containers — контейнеры (образ + изменяемый слой + метаданные)
3. Local Volumes — постоянные тома данных
4. Build Cache — кэш сборки (промежуточные слои)

`{{.Active}}` — количество запущенных/используемых объектов.

Запишите в отчет:

1. Сколько образов (Images) хранится локально ДО начала работы
2. Сколько запущенных контейнеров (Containers) активно сейчас

Задание 1.3 — Артефакт окружения:

```
echo "Student: $(whoami)@$(hostname)" && \  
echo "Date: $(date '+%Y-%m-%d %H:%M:%S')" && \  
docker version --format "Docker {{.Server.Version}} on  
{{.Server.Os}}/{{.Server.Architecture}}"
```

\$(whoami) — подстановка имени текущего пользователя (shell-подстановка).

\$(hostname) — имя хоста машины.

\$(date '+%Y-%m-%d %H:%M:%S') — текущая дата и время в формате ГГГГ-ММ-ДД ЧЧ:ММ:СС.

&& — выполнить следующую команду только если предыдущая завершилась успешно (код 0)

Сделайте скриншот терминала с полным выводом этой команды. На скриншоте должна быть видна командная строка с вашим логином (например, `ivanov@laptop:~$`).

Вопросы:

1. Что произойдёт, если запустить `docker system prune` без флагов? Какие данные будут потеряны?
2. Почему команда `docker version` может завершиться ошибкой, даже если `docker` установлен

Этап 2. Первые шаги: «игрушечные» образы

Задание 2.1 — Запустите классический пример:

```
docker run --rm hello-world
```

docker run — создать контейнер из образа и запустить его.

--rm — автоматически удалить контейнер после завершения (полезно для одноразовых задач).

hello-world — официальный образ из [docker.io/library/hello-world:latest](https://hub.docker.io/library/hello-world).

Что происходит внутри:

Docker проверяет локальный кэш образов.

Если образа нет — скачивает из реестра (по умолчанию Docker Hub).

Создаёт контейнер с изолированной FS.

Запускает бинарник /hello, который выводит сообщение и завершается.

Запишите в отчет:

1. Откуда загружается образ (полное имя репозитория и тег, например `docker.io/library/hello-world:latest`)
2. Какой процесс выполняется внутри контейнера (подсказка: см. вывод)

Задание 2.2 — Исследуйте минимальный образ:

```
docker run --rm alpine:3.18 cat /etc/os-release
```

alpine:3.18 — образ на базе Alpine Linux (размер ~5 МБ), использует пакетный менеджер apk.

cat /etc/os-release — команда, выполняемая внутри контейнера (не на хосте!).

Важно: Контейнер запускает указанную команду как главный процесс (PID 1). После её завершения контейнер останавливается.

Размер образа можно проверить: **docker images alpine:3.18 --format "{{.Size}}"**.

Запишите:

1. Значение поля PRETTY_NAME из вывода
2. Размер образа: `docker images alpine:3.18 --format "{{.Size}}"`

Задание 2.3 — Сравните поведение двух контейнеров:

```
docker run -d --name <фамилия>-alpine-1 alpine:3.18 sleep 300
docker run -d --name <фамилия>-alpine-2 alpine:3.18 sh -c "echo 'Hello from $(hostname)' && sleep 300"
```

-d — detached mode: контейнер работает в фоне, управление возвращается в терминал.

--name — явное имя контейнера (иначе Docker генерирует случайное вида happy_poincare).

sleep 300 — удерживает контейнер в состоянии Up, так как главный процесс не завершается.

sh -c "..." — выполнить несколько команд через оболочку.

\$(hostname) — подстановка выполняется на хосте (до запуска контейнера), так как команда в двойных кавычках.

Проверьте статус:

```
docker ps --filter "name=<фамилия>-alpine"
```

Запишите в отчет скриншот вывода `docker ps` с видимыми именами контейнеров, содержащими вашу фамилию.

Вопросы:

1. Почему образ `hello-world` завершается сразу после запуска, а `alpine:3.18 sleep 300` работает 5 минут?
2. Что произойдёт, если убрать флаг `--rm` из команды `docker run --rm hello-world`? Как удалить «забытые» контейнеры?

Этап 3. Жизненный цикл контейнера

Задание 3.1 — Проследите все состояния через последовательные команды:

```
# 1. Создание без запуска
docker create --name <фамилия>-nginx nginx:alpine

# 2. Запуск
docker start <фамилия>-nginx

# 3. Остановка «мягкая» (сигнал SIGTERM)
docker stop <фамилия>-nginx

# 4. Перезапуск
docker restart <фамилия>-nginx

# 5. Остановка «жёсткая» (сигнал SIGKILL)
docker kill <фамилия>-nginx

# 6. Удаление
docker rm <фамилия>-nginx
```

Важно:

1. nginx:alpine — образ с веб-сервером, который остаётся в фоне (главный процесс не завершается).
2. При docker stop отправляется SIGTERM → процесс может выполнить graceful shutdown (сохранить данные, закрыть соединения).
3. При docker kill отправляется SIGKILL → процесс убивается мгновенно без возможности очистки.

Пояснение: nginx — популярный веб-сервер и прокси. Мы используем официальный образ nginx:alpine, потому что он запускает долгоживущий процесс «из коробки» (в отличие от hello-world, который сразу завершается). Также это реальный инструмент, который вы встретите в продакшене. Вам не нужно понимать, как работает веб-сервер — достаточно знать: контейнер с nginx остается в статусе Up, пока его не остановить.

Задание 3.2 — Зафиксируйте переходы состояний:

```
docker ps -a --filter "name=<фамилия>-nginx" --format "table  
{{.Names}}\t{{.Status}}\t{{.CreatedAt}}"
```

Скопируйте вывод в отчет. Укажите:

1. Какой сигнал отправляет docker stop по умолчанию?
2. Почему docker kill может привести к потере данных в контейнере с БД?

В выводе должен быть виден контейнер в состоянии Exited после kill.
Отчёты без этого состояния — минус 50% за этап.

Вопросы:

Что произойдет, если выполнить docker rm для запущенного контейнера?
Какой флаг нужен для принудительного удаления?

Этап 4. Диагностика реального сервиса

Задание 4.1 — Запуск с ошибкой:

```
docker run -d \  
  --name <фамилия>-pg-broken \  
  -p <ваш_порт>:5432 \  
  postgres:15
```

Объяснение проблемы:

-p <ваш_порт>:5432 — проброс порта хоста → контейнера (5432 — стандартный порт PostgreSQL).

Ошибка: Официальный образ postgres требует обязательную переменную окружения POSTGRES_PASSWORD. Без неё контейнер завершается с ошибкой.

Это пример «безопасности по умолчанию» — СУБД не запустится без пароля.

Диагностика:

```
# Статус контейнера  
docker ps -a --filter "name=<фамилия>-pg"  
  
# Логи с временной меткой (последние 20 строк)  
docker logs --timestamps <фамилия>-pg-broken 2>&1 | tail -20  
  
# Код завершения  
docker inspect <фамилия>-pg-broken --format='{{.State.ExitCode}}'
```

Задание 4.2 — Исправление и проверка:

```
docker run -d \  
  --name <фамилия>-pg-fixed \  
  -e POSTGRES_PASSWORD=Pass_<ваш_порт> \  
  -e POSTGRES_USER=user_<номер_ису> \  
  -p <ваш_порт>:5432 \  
  postgres:15
```

Проверка работоспособности:

```
# Ждём инициализацию БД
sleep 15

# Выполняем запрос к БД
docker exec <фамилия>-pg-fixed psql -U user_<номер_ису> -c "SELECT version();"
```

Задание 4.3 — Финальный “артефакт”:

```
echo "=== Практика №1: $(date '+%Y-%m-%d %H:%M:%S') ===" && \
echo "Студент: $(whoami)@$(hostname)" && \
docker ps --filter "name=<фамилия>-pg-fixed" --format "table
{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Необходимо сделать скриншот терминала с полным выводом этой команды (видны логин, хост, время и статус контейнера).

Требования к отчёту

Файл должен называться: ПР1_Фамилия_ИСУ.pdf (например, ПР1_Иванов_4123.pdf)

Обязательные разделы:

1. Персональные параметры (таблица из начала работы)
2. Скриншоты артефактов выполнения (Этапы 1.3, 2.3, 4.3) — видны логин/хост/время
3. Выводы ключевых команд с краткими пояснениями
4. Таблица состояний контейнера из Задания 3.2